



BEGINNER ONBOARDING / SOLO BUILD FIRST

SubTracker

Beginner Field Guide

Where to start, what to build, what to ignore, and how to get unstuck without waiting for someone else.

First destination

`docs/learning/solo-prototype.md`

Not backend/. Not Docker. Not the whole application.

ONE RULE

Build one small AI loop and be able to explain every part of it.

Your first 30 minutes

1. Read

Open `README.md`, then read `docs/learning/solo-prototype.md` from top to bottom.

2. Restate

In your own words, explain the input, allowed recommendations, required evidence, uncertainty, and savings output.

3. Sketch

Draw this loop: structured input -> Python tools -> model -> Pydantic validation -> evaluation result.

4. Begin

Create the smallest runnable interface: CLI, notebook, or one small FastAPI endpoint.

THE ASSIGNMENT IN ONE SENTENCE

Given subscription and usage evidence, return a validated keep, downgrade, cancel, or review recommendation with evidence, uncertainty, confidence, and deterministic savings.

Permission to ignore

Docker, PostgreSQL, Redis, authentication, a polished frontend, bank connections, cancellation automation, and the complete shared API.

Five steps to a useful prototype

01 Read	Understand the shared task and cases.
02 Model	Define Pydantic input and output schemas.
03 Build	Add a model call and deterministic tools.
04 Break	Test missing data, bad output, and hostile text.
05 Explain	Record decisions, failures, scores, and questions.

REQUIRED INPUT

Subscription and provider; price, currency, and billing period; recent charges; usage events or an explicit unknown state; renewal date; user preferences; plan alternatives; analysis date.

REQUIRED OUTPUT

Action; reason; evidence with references; uncertainties; confidence; monthly and annual savings; a follow-up question when needed; approval-required flag; schema, prompt, and model references.

REQUIRED PYTHON TOOLS

`calculate_savings` computes money and billing periods. `summarize_usage` computes recency and frequency or reports unknown. The model explains verified numbers; it does not invent them.

Read by phase, not by folder size

READ NOW	docs/learning/solo-prototype.md	Your exact task, cases, tools, and completion criteria.
READ NEXT	docs/learning/comparison-and-convergence.md	How all seven experiments will be compared and combined.
READ LATER	docs/product/vision-and-scope.md	The product problem, MVP boundary, and end-state user journey.
READ LATER	docs/architecture/	The shared FastAPI, data, API, and AI-system design.
READ LATER	docs/platform/	Docker, development, security, testing, and operations.
IGNORE FOR SOLO	backend/ and compose.yaml	Shared-system foundation. It is not your first assignment.

WHY EVERYONE BUILDS THE SAME THING

Every participant must personally practice prompting, schemas, tools, grounding, failures, and evaluation. Shared cases make the experiments comparable. The final design is chosen from evidence, not polish.

The full journey

- Phase 0: agree on the same task
- Phase 1: seven solo prototypes
- Phase 2: compare evidence and failures
- Phase 3: build one shared production-structured system

Starting and scope

STARTING**Where do I start?**

Read README.md, then docs/learning/solo-prototype.md. Do not begin in backend/.

STARTING**Do I need to understand the whole repository?**

No. Learn the solo task, its input/output, tools, and cases. Architecture and platform documents are for later phases.

SCOPE**Am I building the entire app?**

No. Build only a narrow recommendation loop. No authentication, full frontend, live bank data, or cancellation automation.

SCOPE**Can I use a CLI or notebook?**

Yes. A CLI, notebook, or one small FastAPI endpoint is enough. AI-system quality matters more than polish.

TOOLS**Do I need Docker or PostgreSQL?**

No. Those are shared-system tools. Your solo time should focus on the AI loop.

TOOLS**Are we using pip?**

No. Use uv for Python dependency management and reproducible commands.

AI concepts in plain language

CONCEPT**What is structured output?**

A predictable data object validated against a schema, not an uncontrolled paragraph. Define it with Pydantic and reject invalid results.

CONCEPT**What is tool calling?**

The model requests a named Python function for a task such as savings calculation. The program runs the tool and returns the verified result.

CONCEPT**What is grounding?**

Every factual claim should be supported by the supplied evidence. The model cannot claim the user stopped using a service when usage is unknown.

CONCEPT**What is an evaluation?**

A repeatable test case with expected allowed actions, required evidence, forbidden claims, savings, and failure severity.

DESIGN**Why not let the model calculate savings?**

Language models are not the source of truth for arithmetic or billing logic. Tested Python code calculates; the model interprets and explains.

SAFETY**What if usage is missing?**

Unknown is not zero. Lower confidence, ask a follow-up question, or return review unless other strong evidence exists.

Failure and troubleshooting

FAILURE**The model returned invalid JSON. Now what?**

Capture the failure, validate the response, retry only within a clear limit, and fail safely when the limit is reached. Record what happened.

FAILURE**The model calculated the wrong savings.**

Move the calculation into `calculate_savings` and pass the verified result into the reasoning context.

FAILURE**The model says cancel when usage is unknown.**

Treat this as a grounding and uncertainty failure. Require acknowledgement of missing data, a question, lower confidence, or review.

QUALITY**My answer sounds good. Is that enough?**

No. Run the shared normal and adversarial cases. Check the schema, facts, calculations, uncertainty, safety, repeatability, latency, and cost.

DONE**How do I know I am done?**

Your prototype handles shared cases and malformed output, uses tested tools, records failures, and you can explain every important design choice.

LEARNING**Should I hide failed cases in my presentation?**

No. Failures are one of the most valuable results. They reveal the checks and architecture the shared system needs.

What happens after the solo build

Every prototype runs the same evaluation cases. The team compares recommendation correctness, grounding, savings accuracy, uncertainty, schema reliability, safety, explanation quality, and maintainability.

Not a winner-takes-all contest

The shared system may use one prototype's schema, another's tools, a third's failure handling, and evaluation ideas from several people.

COMPARISON QUESTIONS

- Which schema represented evidence and uncertainty best?
- Which prompt generalized instead of memorizing examples?
- Which tasks should never have been delegated to the model?
- Which failures could cause the most harm?
- Were confidence values meaningful?
- Which design will be easiest to test and improve?

PHASE 2 OUTPUT

One domain vocabulary, canonical input contract, versioned recommendation schema, selected Python tools, baseline prompt strategy, evaluation dataset, architecture decisions, and shared-system backlog.

Exit rule

Shared product features begin only when the team can defend the shared design using prototype evidence.

A question someone can answer quickly

Before asking, search the interactive FAQ, reread the relevant solo-build section, and reduce the problem to the smallest failing case.

Help template

Case/input: ...
Expected behavior: ...
Actual result/error: ...
Smallest relevant code or prompt: ...
What I already tried: ...
My current theory: ...

NEVER SHARE

API keys, passwords, access tokens, real financial data, or another person's private information.

BEFORE YOU SAY 'I AM STUCK'

1. Read the complete error message.
2. Identify which step failed: input, tool, model call, validation, or evaluation.
3. Run the smallest case again.
4. Print or log the structured values safely.
5. Write what you expected and why.
6. Ask with the template above.

Remember

Start small. Make failures visible. Be able to explain your system.